

LAPORAN PRATIKUM
STRUKTUR DATA
IMPLEMENTASI ADT ARRAYLIST

disusun Oleh:

Muhammad Yasin Habiburrahman

2511532016

Dosen Pengampu:

Dr. Wahyudi. S.T.M.T

Asisten Pratikum:

Sherly Sukmadira Putri



DEPARTEMEN INFORMATIKA FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia-Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman.

Tugas kelompok ini kami susun dengan tujuan untuk memenuhi salah satu tugas mata kuliah Algoritma dan Pemrograman, dengan tema “”, yang mencakup

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pembimbing yang telah memberikan arahan, serta kepada seluruh anggota kelompok yang telah bekerja sama dengan baik sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami khususnya, dan bagi pembaca pada umumnya.

Padang,

Muhammad Yasin Habiburrahman

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN.....	2
2.1 Struktur Dasar ArrayList.....	2
2.2 Kelas Mahasiswa	3
2.3 Kelas DriverMahasiswa	3
2.4 Kelas DaftarKata.....	6
2.5 Kelas DaftarKataDriver	7
BAB III KESIMPULAN.....	9

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak, pengelolaan data secara efisien merupakan hal yang sangat penting. Salah satu cara untuk mewujudkan hal tersebut adalah dengan menggunakan struktur data yang tepat. Abstract Data Type (ADT) merupakan model matematika dari suatu tipe data yang mendefinisikan perilaku dari struktur data tersebut dari sudut pandang pengguna, tanpa memperhatikan detail implementasinya.

ArrayList merupakan salah satu implementasi dari ADT List yang paling banyak digunakan. Berbeda dengan array statis yang ukurannya tetap, ArrayList mampu mengubah ukurannya secara dinamis sesuai kebutuhan. Hal ini menjadikan ArrayList sebagai struktur data yang fleksibel dan efisien untuk menyimpan kumpulan elemen yang jumlahnya tidak dapat ditentukan sejak awal.

1.2 Tujuan

- Memahami konsep Abstract Data Type (ADT) secara umum dan kaitannya dengan struktur data ArrayList.
- Mampu mengidentifikasi dan menjelaskan operasi-operasi dasar pada ADT ArrayList.
- Mampu mengimplementasikan ADT ArrayList menggunakan bahasa pemrograman Java.
- Memahami perbedaan antara array statis dan ArrayList dalam hal pengelolaan memori dan fleksibilitas ukuran.

1.3 Manfaat Praktikum

- Praktikan mampu menerapkan konsep ADT ArrayList dalam menyelesaikan permasalahan pemrograman yang melibatkan pengelolaan data dinamis.
- Praktikan mendapatkan pemahaman yang lebih baik mengenai abstraksi data sebagai dasar perancangan perangkat lunak yang terstruktur.
- Praktikan memiliki keterampilan dalam mengimplementasikan struktur data ArrayList secara mandiri, tidak hanya bergantung pada library bawaan.

BAB II

PEMBAHASAN

2.1 Struktur Dasar ArrayList

```
1 package pekan2_2511532016;
2 import java.util.ArrayList;
3 public class ArrayList1_2511532016 {
4     public static void main(String[] args) {
5         // Size of the ArrayList
6         int n=5;
7         // Declaring the ArrayList with initial size n
8         ArrayList<Integer> arrli = new ArrayList<Integer>(n);
9         // Appending new elements at the end of the list
10        for (int i=1; i<=n; i++) arrli.add(i);
11        // Printing elements
12        System.out.println(arrli);
13        // Remove element at index 3
14        arrli.remove(3);
15        // Displaying the ArrayList
16        // After deletion
17        System.out.println(arrli);
18        // Printing elements one by one
19
20        for(int i =0; i<arrli.size(); i++)
21            System.out.print(arrli.get(i) + " ");
22    }
23 }
```

Kode Program 2.1 – Kode Program ArrayList

Kode program di atas menunjukkan dimulai dengan mendeklarasikan sebuah variabel *n* dengan nilai 5. Ini akan menjadi ukuran dari array list yang akan dibuat. Kemudian, dibuat sebuah objek array list dengan nama *arrli*, dengan tipe data Integer.

Blok program berikutnya dimulai dengan for loop sederhana dari *i*-*n*. Di sini akan dimasukkan angka 1-5 ke dalam *arrli*. Baris berikutnya akan mencetak nilai yang terdapat pada *arrli*. Kemudian, dihapuslah elemen dengan indeks ke-3. Lalu, dicetakkan kembali *arrli* dengan nilai yang sudah dihapus.

Blok program terakhir juga merupakan for loop sederhana dari 0 hingga banyak elemen dari *arrli* saat ini. Karena banyak elemen dari *arli* adalah 4 elemen, maka for loop tersebut akan beriterasi sebanyak 4 kali. Untuk for loop ini, ia akan mengambil semua elemen dari *arrli* menggunakan method *get()*, dan dipisahkan dengan *whitespace* (" ").

Hasil dari program tersebut adalah sebagai berikut.

```
<terminated> Array
[1, 2, 3, 4, 5]
[1, 2, 3, 5]
1 2 3 5
```

Gambar 2.1 – Output dari Kode Program ArrayList1_2511532016

2.2 Kelas Mahasiswa

```
1 package pekan2_2511532016;
2 public class Mahasiswa_2511532016 {
3     String nim;
4     String nama;
5     String prodi;
6     Mahasiswa_2511532016(String nim, String nama, String prodi){
7         this.nim = nim;
8         this.nama = nama;
9         this.prodi = prodi;
10    }
11    @Override
12    public String toString() {
13        return "NIM: " + nim + ", Nama: " + nama + ", Prodi: " + prodi;
14    }
15 }
```

Kode Program 2.2 – Kode Program kelas Mahasiswa

Class Mahasiswa merupakan class model yang merepresentasikan data seorang mahasiswa dengan tiga atribut yaitu nim, nama, dan prodi, ketiganya bertipe String. Class ini memiliki sebuah konstuktur yang menerima ketiga atribut tersebut sebagai parameter dan menginisialisasinya menggunakan keyword “this” untuk membedakan atribut class dengan parameter yang masuk.

Method toString() di-*override* dari class Object bawaan Java agar ketika objek Mahasiswa dicetak, outputnya berupa string yang terformat rapi menampilkan NIM, nama, dan prodi mahasiswa tersebut.

2.3 Kelas DriverMahasiswa

```
// 1. Method untuk menampilkan menu
public static void tampilkanMenu_2511532016() {
    System.out.println("\nMenu:");
    System.out.println("1. Tambah Mahasiswa");
    System.out.println("2. Tampilkan Semua Mahasiswa");
    System.out.println("3. Hapus Mahasiswa Berdasarkan NIM");
    System.out.println("4. Cari Mahasiswa Berdasarkan NIM");
    System.out.println("5. Keluar");
}
```

Kode Program 2.3 – Method untuk Menampilkan Menu

Method ini hanya bertugas mencetak pilihan menu ke layar, mulai dari opsi tambah, tampilkan, hapus, cari, hingga keluar.

```
//2. Method untuk menambahkan mahasiswa
public static void tambahMahasiswa_2511532016(ArrayList<Mahasiswa_2511532016> list, Scanner sc) {
    System.out.print("Masukkan NIM: ");
    String nim = sc.nextLine();
    System.out.print("Masukkan Nama: ");
    String nama = sc.nextLine();
    System.out.print("Masukkan Prodi: ");
    String prodi = sc.nextLine();
    list.add(new Mahasiswa_2511532016(nim, nama, prodi));
    System.out.println("Mahasiswa berhasil ditambahkan!");
}
```

Kode Program 2.4 – Method untuk Menerima Input

Method ini menerima dua parameter yaitu list dan sc. Program meminta input NIM, nama, dan prodi dari pengguna, lalu membuat objek Mahasiswa baru dari ketiga input tersebut dan langsung menambahkannya ke dalam list menggunakan add().

```
//3. Method untuk tampilkan semua data
public static void tampilkanSemuaMahasiswa_2511532016(ArrayList<Mahasiswa_2511532016> list) {
    if(list.isEmpty()) {
        System.out.println("Daftar mahasiswa kosong");
    } else {
        System.out.println("Data Mahasiswa");
        for(Mahasiswa_2511532016 mhs : list) {
            System.out.println(mhs);
        }
    }
}
```

Kode Program 2.5 – Method untuk Menampilkan Semua Data pada Daftar

Sebelum mencetak, method ini mengecek terlebih dahulu apakah list kosong menggunakan isEmpty(). Jika kosong, ditampilkan pesan bahwa daftar kosong. Jika tidak, dilakukan iterasi menggunakan *for-each loop* dan setiap objek akan dicetak. For each loop adalah versi sederhana dari for biasa yang dirancang khusus untuk iterasi koleksi atau array tanpa perlu mengurus indeks secara manual.

```
//4. Method untuk hapus mahasiswa berdasarkan NIM
public static void hapusMahasiswa_2511532016(ArrayList<Mahasiswa_2511532016> list, Scanner sc) {
    System.out.print("Masukkan NIM yang akan dihapus: ");
    String nimHapus = sc.nextLine();
    boolean removed = list.removeIf(mhs -> mhs.nim.equals(nimHapus));
    if (removed) {
        System.out.println("Data dengan NIM " + nimHapus + " berhasil dihapus.");
    } else System.out.println("NIM tidak ditemukan.");
}
```

Kode Program 2.6 – Method untuk Menghapus Mahasiswa Berdasarkan NIM

`mhs -> mhs.nim.equals(nimHapus)` merupakan contoh ekspresi lambda pada Java. Ekspresi ini mempersingkat penulisan for loop menjadi satu baris program saja. Versi for loop normalnya adalah sebagai berikut.

```
for (int i = 0; i < list.size(); i++) {
    if (list.get(i).nim.equals(nimHapus)) {
        list.remove(i);
        break;
    }
}
```

```
}
```

Method ini menggunakan `removeIf()` dengan lambda expression `mhs -> mhs.nim.equals(nimHapus)` untuk menghapus elemen yang NIM-nya cocok. `removeIf()` mengembalikan nilai boolean — true jika ada elemen yang berhasil dihapus, false jika tidak ditemukan — sehingga bisa langsung dipakai untuk menampilkan pesan hasil operasi.

```
//5. Method untuk cari mahasiswa berdasarkan NIM
public static void cariMahasiswa_2511532016(ArrayList<Mahasiswa_2511532016> list, Scanner sc) {
    System.out.print("Masukkan NIM yang dicari: ");
    String nimCari = sc.nextLine();
    boolean ditemukan = false;
    for(Mahasiswa_2511532016 mhs: list) {
        if(mhs.nim.equals(nimCari)) {
            System.out.println("Hasil Pencarian: " + mhs);
            ditemukan = true;
            break;
        }
    }
    if(!ditemukan) {
        System.out.println("NIM tidak ada.");
    }
}
```

Kode Program 2.7 – Method untuk Mencari Mahasiswa Berdasarkan NIM

Method ini melakukan pencarian dengan iterasi satu per satu menggunakan *for each loop*, membandingkan NIM tiap objek dengan NIM yang dicari menggunakan `equals()`. Jika ditemukan, data langsung dicetak dan perulangan dihentikan dengan `break`. Variabel “ditemukan” digunakan sebagai flag untuk mendeteksi kasus NIM tidak ada di list.

```
public static void main(String[] args) {
    ArrayList<Mahasiswa_2511532016> mahasiswaList = new ArrayList<>();
    Scanner scanner = new Scanner(System.in);
    int choice;
    do {
        tampilkanMenu_2511532016();
        System.out.print("Pilih menu: ");
        choice = scanner.nextInt();
        scanner.nextLine();

        switch(choice){
            case 1:
                tambahMahasiswa_2511532016(mahasiswaList, scanner);
                break;
            case 2:
                tampilkanSemuaMahasiswa_2511532016(mahasiswaList);
                break;
            case 3:
                hapusMahasiswa_2511532016(mahasiswaList, scanner);
                break;
            case 4:
                cariMahasiswa_2511532016(mahasiswaList, scanner);
                break;
            case 5:
                System.out.println("Keluar dari program.");
                break;
            default:
                System.out.println("Pilihan tidak valid");
        }
    } while (choice != 5);
    scanner.close();
}
```

Kode Program 2.8 – Method Utama

Program berjalan dalam struktur *do-while* yang terus mengulang selama input yang diberikan tidak sama dengan 5. Setiap iterasi menampilkan menu, membaca pilihan dengan `nextInt()`, lalu `scanner.nextLine()` dipanggil setelahnya untuk membersihkan sisa newline di buffer agar input String berikutnya tidak terganggu. Pilihan kemudian diteruskan ke `switch` yang memanggil method yang sesuai. Saat

pengguna memilih 5, loop berhenti dan scanner.close() dipanggil untuk menutup resource.

2.4 Kelas DaftarKata

```
public class DaftarKata_2511532016 {
    private final ArrayList<String> data;

    // Konstruktor: inisialisasi list kosong
    public DaftarKata_2511532016() {
        this.data = new ArrayList<>();
    }
}
```

Kode Program 2.9 – Inisialisasi Array List

Class ini memiliki satu atribut, yaitu “data”, bertipe ArrayList<String> yang dideklarasikan private final. private berarti atribut ini tidak bisa diakses langsung dari luar class, sedangkan final berarti referensi data tidak bisa diganti ke objek ArrayList lain setelah diinisialisasi. Konstruktor menginisialisasi data sebagai list kosong.

```
//Menambahkan elemen di akhir list
public void tambah_2511532016(String elemen) {
    data.add(elemen);
}
//Menambahkan elemen pada indeks tertentu
public void tambahPada_2511532016(int index, String elemen) {
    data.add(index, elemen);
}
```

Kode Program 2.10 – Menambahkan Elemen ke List

Keduanya wrapper dari method add() milik ArrayList. Wrapper adalah method yang "membungkus" method lain tanpa menambah logika. Tujuannya cuma untuk menyembunyikan detail implementasi dari pengguna class. tambah() menambahkan elemen di akhir list, sedangkan tambahPada () menerima parameter indeks tambahan untuk menyisipkan elemen di posisi tertentu. Elemen yang sudah ada di indeks tersebut dan seterusnya otomatis bergeser ke kanan.

```
public void ubahElemen_2511532016(int index, String nilaiBaru) {
    data.set(index, nilaiBaru);
}
```

Kode Program 2.11 – Mengubah Elemen pada Indeks Tertentu

Menggunakan set(index, nilaiBaru) untuk mengganti elemen pada indeks tertentu dengan nilai baru. Berbeda dengan add() yang menyisipkan, set() menimpa elemen yang sudah ada tanpa menggeser elemen lain.

```
public String hapusElemen2511532016(int index) {
    return data.remove(index);
}
```

Kode Program 2.12 – Menghapus Elemen pada Indeks Tertentu

Memanggil `remove(index)` dan mengembalikan elemen yang dihapus sebagai String. Jadi selain menghapus, method ini juga memberi tahu pemanggil elemen mana yang baru saja dihilangkan dari list.

```
public void iterasiCetak_2511532016() {
    for (int i=0; i< data.size(); i++) {
        System.out.print(data.get(i)+ " ");
    }
}
```

Kode Program 2.13 – Menampilkan Semua Elemen

Mencetak semua elemen list secara menggunakan for biasa dengan indeks. Menggunakan `data.size()` sebagai batas agar iterasi selalu menyesuaikan ukuran list yang bersifat dinamis.

```
public String get(int index) {
    return data.get(index);
}

@Override
public String toString() {
    return data.toString();
}
```

Kode Program 2.14 – Mengambil Elemen pada Indeks Tertentu; Mengubah Array List Menjadi Tipe Data String

`get()` adalah getter sederhana untuk mengakses elemen di indeks tertentu dari luar class. `toString()` di-override untuk mendelegasikan representasi String-nya langsung ke `data.toString()` milik `ArrayList`.

2.5 Kelas DaftarKataDriver

```
1 package pekan2_2511532016;
2
3 public class DaftarKataDriver_2511532016 {
4
5     public static void main(String[] args) {
6         DaftarKata_2511532016 al = new DaftarKata_2511532016();
7
8         // Menambah elemen (akhir)
9         al.tambah_2511532016("Kami ");
10        al.tambah_2511532016("Informatika");
11
12        // Menyisipkan elemen pada indeks 1
13        al.tambahPada_2511532016(1, "Mahasiswa");
14
15        // Cetak isi awal
16        System.out.println("Awal : " + al);
17
18        // Mengubah elemen (index 1)
19        al.ubahElemen_2511532016(1, "Departemen");
20        System.out.println("Setelah Ubah: " + al);
21
22        // Menghapus elemen (hapus index 0)
23        String terhapus = al.hapusElemen2511532016(0);
24        System.out.println("Terhapus : " + terhapus);
25        System.out.println("Setelah Hapus: " + al);
26
27        //Iterasi pada ArrayList
28        System.out.print("Iterasi:");
29        al.iterasiCetak_2511532016();
30        System.out.println();
31
32
33    }
34
35 }
```

Kode Program 2.15 – Program Driver untuk Mengetes Kelas DaftarKata

Pertama, dua elemen ditambahkan di akhir list — "Kami " dan "Informatika" — sehingga list menjadi [Kami, Informatika], lalu "Mahasiswa" disisipkan di indeks 1 sehingga list menjadi [Kami, Mahasiswa, Informatika] dan dicetak sebagai kondisi awal. Selanjutnya elemen di indeks 1 diubah dari "Mahasiswa" menjadi "Departemen" menggunakan ubahElemen(), lalu elemen di indeks 0 yaitu "Kami " dihapus menggunakan hapusElemen() — nilai yang terhapus ditangkap ke variabel terhapus dan dicetak tersendiri, sehingga list akhir menjadi [Departemen, Informatika]. Terakhir, isi list dicetak ulang menggunakan iterasiCetak() yang mencetak tiap elemen satu per satu secara horizontal.

BAB III

KESIMPULAN

ArrayList merupakan implementasi dari ADT List yang bersifat dinamis, artinya ukurannya dapat bertambah atau berkurang secara otomatis sesuai kebutuhan, berbeda dengan array statis yang ukurannya harus ditentukan sejak awal.

ArrayList menyediakan berbagai operasi dasar seperti penambahan elemen di akhir list maupun pada indeks tertentu (`add()`), pengubahan elemen (`set()`), penghapusan elemen (`remove()`), dan pengaksesan elemen (`get()`), yang semuanya dapat digunakan secara langsung maupun dibungkus dalam method custom melalui prinsip enkapsulasi.

Konsep enkapsulasi diterapkan dengan menjadikan atribut ArrayList sebagai `private` sehingga akses dan manipulasi data hanya dapat dilakukan melalui method yang telah disediakan, menjadikan kode lebih terstruktur dan mudah dipelihara. Selain itu, ArrayList dapat menyimpan objek dari class buatan sendiri seperti Mahasiswa, sehingga sangat fleksibel untuk merepresentasikan data yang lebih kompleks dalam sebuah program. Secara keseluruhan, pemahaman terhadap ADT ArrayList memberikan fondasi yang kuat dalam merancang struktur data yang efisien dan berorientasi objek.